Code

Learn code for
Steps for:
1. Practicals
2. Functions
3. Applications
4. Best practices
5. Essential Steps and their purpose
6. Additional steps

Learn code for

1. histograms
2. scatterplot
3. boxplot
4. violin plot
5. pairplot
6. 3d scatter plot
7. correlation matrix
8. uploading files, loading them from file path

Steps for:

- basic data prep:
  1. Import statements
  2. Ingest/Load data opt. Convert to df
  3. Display
- ML:
  1. Import statements
  2. Ingest/Load data
  3. EDA
  4. Split train and test
  5. Scaling
  6. train
  7. test
  8. Evaluate performance
- Linear Regression:
  1. import libraries
  2. load dataset
     a. initialize x and y
     b. EDA (optional)
     c. Scaling (optional)
  3. split
  4. initialize model
  5. train
  6. make predictions
  7. evaluate the model
  8. plot results

1. Practicals

▼ 2.1: Basic Statistics with Python (https://colab.research.google.com/drive/1T0cV05XXKBQ4EPloS-RuHz8A1qk8lgx1)

▼ 1. To import dataset

```
import numpy as np
from sklearn.datasets import load_wine #import dataset
import pandas as pd
```

Imports the NumPy library, which is a powerful numerical computing library in Python. It's often used for working with arrays and matrices, performing mathematical operations, and providing support for large, multi-dimensional data.

Here, you're importing the `load_wine` function from the `datasets` module of the scikit-learn library (`sklearn`). This function loads the Wine dataset.

Import the Pandas library, which is widely used for data manipulation and analysis. It provides data structures like DataFrames, which are very useful for handling structured data (like tables).

▼ 2. To load the dataset

```
wine_data = load_wine()
wine_df = pd.DataFrame(wine_data.data, columns=wine_data.feature_names)
```

`load_wine()` loads the Wine dataset from scikit-learn.

`pd.DataFrame()` is used to create a DataFrame from the data.

`wine_data.data`: The NumPy array containing the feature values (the various chemical properties).

`columns=wine_data.feature_names`: This argument sets the column names of the DataFrame to the feature names provided in the dataset.

▼ 3. Statistical measurements

```
mean_values = wine_df.mean()
median_values = wine_df.median()
std_values = wine_df.std()
variance_values = wine_df.var()
min_values = wine_df.min()
max_values = wine_df.max()
skewness_values = wine_df.skew()
kurtosis_values = wine_df.kurt()
```

▼ 4. Summarizing Statistics

Summarizing All Statistics After calculating all the statistics, we can create a DataFrame to display the results in a more organized manner:

```
# Create a DataFrame to show the results
stats_summary = pd.DataFrame({
    'Mean': mean_values,
    'Median': median_values,
    'Standard Deviation': std_values,
    'Variance': variance_values,
    'Min': min_values,
    'Max': max_values,
    'Skewness': skewness_values,
    'Kurtosis': kurtosis_values
```

```
})

# Display the statistics in the console
print(stats_summary)
```

▼ Conclusion

In this tutorial, you learned how to calculate various statistics using pandas with the Wine dataset from sklearn. These statistics are essential for understanding the central tendency, variability, and distribution characteristics of the data. By performing these operations, you gain valuable insights into the dataset that help in further analysis.

▼ 2.2: Statistics using NumPy (https://colab.research.google.com/drive/1jtmIfQb1dIQDGyuQFsK9IueSYFH7EcE2)

▼ 1. Import Libraries

```
import numpy as np
from sklearn.datasets import load_wine
```

▼ 2. Load the Dataset

```
# Load Wine dataset
wine = load_wine()
```

▼ 3. Extract features

```
# Extract the features (data) and target (labels)
X = wine.data
y = wine.target
```

- `X = wine.data` assigns the features of the dataset (the chemical properties) to the variable `X`. This variable is commonly used to represent input data in machine learning.
- `y = wine.target` assigns the target labels (the classes of wine) to the variable `y`. This variable is used to represent the output or labels that you want to predict.

▼ 4. Statistical measurements

Minimum and maximum

```
max_features = np.max(X, axis=0)
min_features = np.min(X, axis=0)
print("\nMaximum value of each feature:\n", max_features)
print("\nMinimum value of each feature:\n", min_features)
```

Median

```
median_features = np.median(X, axis=0)
print("\nMedian of each feature:\n", median_features)
```

Variance

```
variance_features = np.var(X, axis=0)
print("\nVariance of each feature:\n", variance_features)
```

25th and 75th percentiles (interquartile range)

```
percentile_25 = np.percentile(X, 25, axis=0)
percentile_75 = np.percentile(X, 75, axis=0)
print("\n25th percentile of each feature:\n", percentile_25)
print("\n75th percentile of each feature:\n", percentile_75)
```

Range

```
range_features = max_features - min_features
print("\nRange of each feature (max - min):\n", range_features)
```

Skewness

```
skewness_features = np.mean(((X - np.mean(X, axis=0)) / np.std(X, axis=0))**3, axis=0)
print("\nSkewness of each feature:\n", skewness_features)

#axis=0 the 1st column
#**3 = to the power of 3

(x-mean)/std(x)**3
```

Kurtosis

```
kurtosis_features = np.mean(((X - np.mean(X, axis=0)) / np.std(X, axis=0))**4, axis=0) - 3
print("\nKurtosis of each feature:\n", kurtosis_features)
```

Proportion of each class in the target

```
unique, counts = np.unique(y, return_counts=True)
proportion_classes = counts / len(y)
print("\nProportion of each class in the target:\n", dict(zip(unique, proportion_classes)))
```

Cumulative sum for each feature

```
cumsum_features = np.cumsum(X, axis=0)
print("\nFirst 5 rows of cumulative sum for each feature:\n", cumsum_features[:5])
```

Geometric mean

```
geometric_mean_features = np.exp(np.mean(np.log(X + 1e-10), axis=0))  # Added small value to avoid log(0)
print("\nGeometric mean of each feature:\n", geometric_mean_features)
```

Harmonic mean

```
harmonic_mean_features = len(X) / np.sum(1.0 / (X + 1e-10), axis=0)  # Added small value to avoid division by
print("\nHarmonic mean of each feature:\n", harmonic_mean_features)
```

- `X`: This is the 2D NumPy array containing the features of the Wine dataset.
- `axis=0`: This parameter specifies that the operation should be performed along the rows (i.e., for each column, or feature). The result is a 1D array containing the values of each feature.

▼ 2.3: Statistics using Pandas (https://colab.research.google.com/drive/1uKcDZwpfxyn_pGShFA2QJaT1DBE2n3ok)

▼ Step 1 : Import libraries

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_diabetes
```

▼ Step 2 : Load the dataset

```
diabetes = load_diabetes()
df = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
df['target'] = diabetes.target
```

`df['target'] = diabetes.target` adds a new column to the DataFrame named 'target', which contains the target values from the dataset (typically representing the disease progression measure).

▼ Step 3 : Display the first few rows of the dataset

```
# Display the first few rows of the dataset
# A quick look at the dataset structure
print(df.head())
```

▼ Step 4 : Statistical measurements

- computes for each column in the DataFrame `df.`. Since `df.` contains both the feature data and the target variable, this function will return the values for each feature (columns) in the dataset. The result of each is assigned to their variable.

Mean

```
mean_features = df.mean()
print("\nMean of each feature:\n", mean_features)
```

Standard deviation

```
std_features = df.std()
print("\nStandard deviation of each feature:\n", std_features)
```

Median

```
median_features = df.median()
print("\nMedian of each feature:\n", median_features)
```

Maximum and Minimum

```
max_features = df.max()
min_features = df.min()
print("\nMaximum value of each feature:\n", max_features)
print("\nMinimum value of each feature:\n", min_features)
```

Variance

```
variance_features = df.var()
print("\nVariance of each feature:\n", variance_features)
```

25th and 75th percentile

```
percentile_25 = df.quantile(0.25)
percentile_75 = df.quantile(0.75)
print("\n25th percentile of each feature:\n", percentile_25)
print("\n75th percentile of each feature:\n", percentile_75)
```

Range

```
range_features = df.max() - df.min()
print("\nRange of each feature (max - min):\n", range_features)
```

Skewness

```
skewness_features = df.skew()
print("\nSkewness of each feature:\n", skewness_features)
```

Kurtosis

```
kurtosis_features = df.kurt()
print("\nKurtosis of each feature:\n", kurtosis_features)
```

Correlation between each feature and the target variable

```
correlation_with_target = df.corr()['target'].sort_values(ascending=False)
print("\nCorrelation of each feature with the target variable:\n", correlation_with_target)
```

Cumulative sum of each feature

```
cumsum_features = df.cumsum()
print("\nFirst 5 rows of cumulative sum for each feature:\n", cumsum_features.head())
```

▼ 3.1: Basic Data Prep and Pre-Processing (https://colab.research.google.com/drive/1HH3S8th-0Xoj0bChSxWxd3JLhKjhHyuS)

▼ Step 1 : Import libraries

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

▼ Step 2: Load and explore the dataset

```
iris = load_iris()

iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
iris_df['target'] = iris.target

iris_df.head()
```

▼ Step 3 : Data Preprocessing

```
# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(iris_df.iloc[:, :-1], iris_df['target'], test_size=0.3, random_state=4
```

Check the shape of the training data:

```
print("Training data shape:", X_train.shape) print("Testing data shape:", X_test.shape)
```

Standardize features using standard scaler

```
# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

▼ Step 4 : Train Logistic Regression Model

```
# Initialize and train the Logistic Regression model
model = LogisticRegression(max_iter=200)
model.fit(X_train_scaled, y_train)
```

Display model coefficients

```
print("Model coefficients:", model.coef_)
```

▼ Step 5: Evaluate the model

1. Predict the target values for the testing set using the trained model.
2. Evaluate the model's performance using various metrics like accuracy, confusion matrix, and classification repo

```
# Predict on the test set
y_pred = model.predict(X_test_scaled)
```

```
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy score:", accuracy)

# Display confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("
Confusion Matrix:
", conf_matrix)

# Classification report for precision, recall, and F1-score
class_report = classification_report(y_test, y_pred)
print("
Classification Report:
", class_report)
```

▼ Step 6 : Visualize results

Visualize the confusion matrix using a heatmap to get a better understanding of the model's classification performance.

```
# Visualize the confusion matrix
plt.figure(figsize=(6,4))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

Plot the distribution of target values in the Iris dataset to see how the classes are distributed.

```
# Visualize the distribution of target values
sns.countplot(x=iris_df['target'])
plt.title('Distribution of Target Values in Iris Dataset')
plt.show()
```

▼ Step 7 : Test with another dataset

```
# Load the Wine dataset
wine = load_wine()
wine_df = pd.DataFrame(data=wine.data, columns=wine.feature_names)
wine_df['target'] = wine.target
```

▼ 3.2: Physical File and Missing Values (https://colab.research.google.com/drive/1Q6tFY8Yk2IiL1F5e3e3C9a7tkiY1tRSU)

▼ Step 1 : Import files

```
from google.colab import files

# Upload the CSV file
uploaded = files.upload()
```

▼ Step 2 : Load and explore the dataset

```
import pandas as pd

# Load the CSV file from the uploaded file
file_path = list(uploaded.keys())[0]  # Retrieve the file path from the uploaded file

# Read the CSV file into a pandas DataFrame
df = pd.read_csv(file_path)

# Display the first few rows of the DataFrame
print("First five rows of the dataset:")
print(df.head())
```

▼ Step 3 : Display summary statistics

```
# Display summary statistics
print("\nStatistical summary of the dataset (numerical columns):")
print(df.describe())
```

▼ Step 4 : Identify Missing Values

```
# Check for missing values
missing_values = df.isnull().sum()

# Display columns with missing values
print("\nMissing values in each column:")
print(missing_values[missing_values > 0])
```

▼ Step 5 : Calculate Missing Value Percentage

```
# Calculate and display the percentage of missing values for each column
missing_percentage = (df.isnull().sum() / len(df)) * 100

# Display the percentage of missing values for columns that have missing data
print("\nPercentage of missing values in each column:")
print(missing_percentage[missing_percentage > 0])
```

▼ 4.1: EDA (https://colab.research.google.com/drive/1lYnGrQnchvmN0-BNICrmqmrOSyK228MF)

▼ Step 1 : Import libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_diabetes
```

▼ Step 2 : Data Loading and Preliminary Inspection

```
from google.colab import files
from os.path import exists
uploaded = files.upload()
```

▼ Step 3 : Load the dataset

```
diabetes_data = load_diabetes()
df = pd.DataFrame(data=diabetes_data.data, columns=diabetes_data.feature_names)
df['Disease Progression'] = diabetes_data.target  # Adding the target variable
```

▼ Step 4 : Explore the dataset

```
print("Shape of DataFrame:", df.shape)
print("First 5 rows of the DataFrame:")
print(df.head())
print("Last 5 rows of the DataFrame:")
print(df.tail())
```

▼ Step 5 : Data types and missing value

```
print("Data types and missing values:")
print(df.info())
```

▼ Step 6: Descriptive statistics

```
print("Statistical summary of the features and target:")
print(df.describe())
```

▼ Step 7 : Data distribution visualization

```
# Selecting a subset of features for histograms
features_for_histogram = ['bmi', 'bp']
df[features_for_histogram].hist(bins=15, figsize=(15, 5), layout=(1, 2))
plt.show()
```

- **features_for_histogram**: This is a list containing the names of the columns you want to visualize as histograms. In this case, the columns are `'bmi'` and `'bp'`.
- **df[features_for_histogram]**: This selects the specified columns (`'bmi'` and `'bp'`) from the DataFrame `df`.
- **.hist()**: This method creates histograms for the selected features.

- **bins=15**: This argument specifies that each histogram should use 15 bins, which will determine how the data is grouped for display.
- **figsize=(15, 5)**: This sets the size of the figure to 15 inches wide and 5 inches tall, allowing for a clear view of both histograms.
- **layout=(1, 2)**: This defines the layout of the plots, indicating that there will be 1 row and 2 columns (i.e., the histograms will be placed side by side).

▼ Step 8 : Simple visualization of relationships

Scatter plots

- **plt.figure(figsize=(12, 5))**: This initializes a new figure for plotting and sets its size to 12 inches wide and 5 inches tall.
- **plt.subplot(1, 2, 1)**: This creates a subplot in a 1-row, 2-column layout, placing the first scatter plot in the first position.
- **sns.scatterplot(x='bmi', y='Disease Progression', data=df)**: This uses Seaborn's `scatterplot` function to plot `bmi` on the x-axis and `Disease Progression` on the y-axis, using the data from the DataFrame `df`.
- **plt.title('BMI vs Disease Progression')**: This sets the title for the first subplot.
- **plt.subplot(1, 2, 2)**: This prepares the second subplot in the same 1-row, 2-column layout.
- **sns.scatterplot(x='bp', y='Disease Progression', data=df)**: This creates the second scatter plot, plotting `bp` on the x-axis and `Disease Progression` on the y-axis.

```
# Scatter plots for 'BMI' vs 'Disease Progression' and 'BP' vs 'Disease Progression'
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.scatterplot(x='bmi', y='Disease Progression', data=df)
plt.title('BMI vs Disease Progression')

plt.subplot(1, 2, 2)
sns.scatterplot(x='bp', y='Disease Progression', data=df)
plt.title('BP vs Disease Progression')
plt.show()


# Select 3 columns for the pairplot
selected_columns = df[['bmi', 'bp', 'Disease Progression']]

# Create a pairplot using seaborn
sns.pairplot(selected_columns)
plt.show()


#Correlation matrix
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Matrix')
plt.show()



# Selecting a subset of features for histograms
subset_features = ['mean radius', 'mean texture', 'mean smoothness', 'mean symmetry']
df[subset_features].hist(bins=15, figsize=(15, 6), layout=(2, 2))
```

```
plt.show()



# Box Plot for Multiple Features
plt.figure(figsize=(10, 6))
sns.boxplot(data=df[['bmi', 'bp', 's1', 's2', 's3']])
plt.title('Box Plot of Selected Features')
plt.ylabel('Values')
plt.show()



# Violin Plot for Selected Features
plt.figure(figsize=(10, 6))
sns.violinplot(data=df[['bmi', 'bp', 's1', 's2', 's3']])
plt.title('Violin Plot of Selected Features')
plt.ylabel('Values')
plt.show()



#3d scatter plot

from sklearn.decomposition import PCA

# 3D Scatter Plot of three variables: BMI, Blood Pressure (bp), and Disease Progression (target)

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
sc = ax.scatter(df['bmi'], df['bp'], df['Disease Progression'], c=df['Disease Progression'], cmap='viridis', s=50

ax.set_xlabel('BMI')
ax.set_ylabel('Blood Pressure (BP)')
ax.set_zlabel('Disease Progression')
plt.title('3D Scatter Plot of BMI, BP, and Disease Progression')
plt.colorbar(sc, label='Disease Progression')
plt.show()
```

▼ Step 9 : Simple visualization of class distribution

```
# Adding a 'diagnosis' column to the DataFrame
df['diagnosis'] = data.target

# Mapping target to actual labels for better understanding
df['diagnosis'] = df['diagnosis'].map({0: 'malignant', 1: 'benign'})

# Plotting the distribution of benign and malignant cases
sns.countplot(x='diagnosis', data=df)
plt.title('Distribution of Benign and Malignant Cases')
plt.show()
```

▼ 5.1: ML - Simple Classification (https://colab.research.google.com/drive/1sl6Wi8seQe5vsiGxmd0F92FQ2RKXxRy0)

▼ 5.2: ML2 (https://colab.research.google.com/drive/1HI4Zguz70CDPoQLx0ehMIspcnMP66H3d)

▼ 5.3: ML3 (https://colab.research.google.com/drive/1vBZqXZDH3gJ8d2Fwwrb0dcV3TvXzr2tT)

▼ 6.1: Simple Linear Regression (https://colab.research.google.com/drive/1yhS89Ceg9lLYxgqI8xxktjmCiMgVcTjq)

▼ 6.2: Multiple Linear Regression (https://colab.research.google.com/drive/1iiDIo0BS0piLmMCtOHyviBzH7XAfbgTO)

▼ 7: KNN Classification (https://colab.research.google.com/drive/1m_Z43GrVg5KrzA8FrLjl4tC9DBPZUFvS)

▼ 9.1: K-Means Clustering - with elbow (https://colab.research.google.com/drive/1qLvUsVlJUdcpn-3Y7gpvryAreY27R9IM)

▼ 9.2: K-Means Clustering - without elbow (https://colab.research.google.com/drive/1BejROAjMbm7YGJDL_vApv2KwBBPOVQZo)

▼ 10.1: Feature Selection - Chi2Score (https://colab.research.google.com/drive/1iPqzRWNRyj7_jY8Gst3Bd70BDNcAQK19)

▼ 10.2: Feature Importance (https://colab.research.google.com/drive/1eve6oVJNkLoqK_2IcKstEyXzsOTomsxi#scrollTo=fqoM2mxX8ODi)

▼ 11.1: Feature Selection - Wrapper method (https://colab.research.google.com/drive/1cK4sJ3BaEAMsVGddit6KJeJawFWgdYWB)

▼ 11.2: Feature Selection - Filter method (https://colab.research.google.com/drive/1FSwaWxF8jfb4pIrwEiH38aKMjiAilzu9)

▼ Visualization 1 (https://colab.research.google.com/drive/19FI3a6v0qp1bzX-GsDY45hHbs5s82DHP)

▼ Visualization 2 (https://colab.research.google.com/drive/1dCCpftDYIBdMB-nrL6N5eWeGykDpKO6E)

▼ Dataset Staff Information (https://drive.google.com/file/d/16KvpvZ3N4W7qTmL7C5aDLH_iEECpkeS7/view?usp=sharing)

▼ Decision Tree with and without hypertune (https://colab.research.google.com/drive/17FUb3o2KcDI02gZqTyDBmnPp44EuFnQq)

▼ Graph 1 (https://colab.research.google.com/drive/12z0VvPd6O-CQ76aZWH_RvSgNDmS3NBvQ)

▼ Graph 2 (https://colab.research.google.com/drive/190_nUNrptfaZciofFtXDRJ_hKAH0q-Q8)

2. Functions

▼ df.describe()

gives description of data - summary of statistics in tabular format

can be used for data inspection

manipulates the data frame

but only for numerical

so alphanumerical data must be massaged or fixed (workaround)

▼ df.info()

data types and missing values

3. Applications

▼ Find missing values

isnull(), shape(), df.info

▼ Data inspection

can be done using: df.describe function, box plots etc.

4. Best practices

▼ 1. Split the code

reduces execution time and allows to focus on other things

ex: import statements in one cell

5. Essential Steps and their purpose

▼ Import

python only recognizes certain available libraries when imported

libraries for data manipulation, ML agorithms, formatting, visualization.

▼ Load dataset and convert to data frame

import the dataset by putting it into variable

datasets are in different formats

we put it into a dataframe

dataframes are flexible for data manipulation

takes data from the data column and features from features column

6. Additional steps

▼ Merging datasets

datasets can be in different formats and sizes and can be merged together for more accurate analysis.

Pandas is easier than numpy and more commonly used.

As numpy is mainly used in other types of statistics.